

Namo Roadmap

Design philosophy

Namo's foundational insight is that every key in a hash is a dimension. There is no distinction between "axes" and "values" — a hash `{symbol: 'BHP', date: '2025-01-01', close: 42.5}` has three dimensions, and all three have coordinates. `symbol` has coordinate 'BHP', `date` has coordinate '2025-01-01', `close` has coordinate 42.5.

This means coordinates are values and values are coordinates. You can select on `close` the same way you select on `symbol` — `namo[close: 42.5]` is just as valid as `namo[symbol: 'BHP']`. You can project to `symbol` and `date` and drop `close`, or project to `close` alone. No dimension is privileged. No field is "just data."

This simplification is powerful because it eliminates the distinction that every other tool forces you to make at ingestion time. Pandas requires you to decide which columns are the index and which are data. xarray requires you to declare dimensions, coordinates, and data variables separately. Quantrix requires you to define categories and items. In each case, the user must understand their data's structure before they can load it.

Namo infers everything from the hash keys. The structure is the data. There is no configuration step, no reshaping, no schema declaration. An array of hashes from a database, a CSV, a JSON API, or a YAML file goes straight into Namu and every key is immediately selectable, projectable, and usable in formulae.

This also means that formulae — which create new named computations — are indistinguishable from data dimensions. `row.close` and `row.earnings_yield` resolve through the same mechanism. The consumer doesn't know or care whether a dimension is stored data or a computed formula. They're all just names with values.

Type agnosticism

Namo is not a computation engine or a multidimensional spreadsheet. It is a multidimensional database. The distinction matters: spreadsheets and numerical tools like Quantrix, Improv, xarray, Pandas, and Polars are fundamentally organised around numbers. Their dimensions exist to label numerical data. Text in a dimension is a category label, not something you compute on.

Namo doesn't privilege numbers. A formula can concatenate strings, produce status labels, or generate alert messages as naturally as it can compute ratios:

```
e.label  = proc{ "#{symbol} ({exchange})" }
e.status = proc{ volume > 0 ? 'active' : 'suspended' }
e.alert  = proc{ "#{symbol}: #{action} at #{close}" }
```

These are just as valid as `proc{ close / book_value }`. The formula mechanism resolves names and calls procs. What the proc does with the values — arithmetic, string manipulation, date logic, pattern matching — is Ruby's business, not Namu's.

This means Namu can handle datasets that aren't numerical at all: customer records, event logs, text corpora, legal documents with categorised metadata, survey responses. Anything expressible as an array of hashes. The

selection, projection, contraction, composition, and set operators all work on text, dates, booleans, and arbitrary objects the same way they work on numbers. `namo[status: 'active']` works because `status` is a dimension like any other, and strings are values like any other. The selection mechanism applies to all types equally — not because it ignores type, but because it doesn't restrict it.

No other tool in this space offers this. Pandas comes closest — DataFrames can hold mixed types — but its computation model (`df['col'].rolling().mean()`) assumes numeric columns. `xarray` is explicitly numerical. `Quantix` and `Improv` are spreadsheets. `Polars` is a columnar computation engine optimised for numeric and string operations but not arbitrary Ruby objects. `Namo`'s formula mechanism works on whatever Ruby can work on, which is everything.

Current state: 0.5.0

0.0.0 (2026-03-15): Initial release

Instantiation from an array of hashes. `Namo` infers dimension names from hash keys and extracts unique coordinate values per dimension automatically. Selection via keyword arguments in `[ ]`, supporting single values, arrays, and ranges.

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

sales.dimensions    # => [:product, :quarter, :price, :quantity]
sales.coordinates   # => {product: ['Widget', 'Gadget'], quarter: ['Q1', 'Q2'], price: [10.0, 25.0], quantity: [100, 150, 40]}
sales[product: 'Widget']      # single value
sales[quarter: ['Q1', 'Q2']]  # array
sales[price: 10.0..20.0]      # range
```

This release established the foundational insight: every key is a dimension, every value is a coordinate. No configuration, no schema, no reshaping step.

0.1.0 (2026-03-28): Formulae and projection

Formulae via `[ ]=`. A formula is a proc assigned to a name. It receives a `Row` object and resolves named references against row data or other formulae. Formulae compose — a formula can reference another formula, and the dependency chain resolves lazily through `Row`.

Projection via positional symbol arguments in `[ ]`. Selection and projection can be combined in a single call or chained.

```

sales[:revenue] = proc{|r| r[:price] * r[:quantity]}
sales[:profit] = proc{|r| r[:revenue] - r[:cost]}

sales[:product, :quarter, :revenue]          # projection
sales[:product, :revenue, product: 'Widget'] # projection + selection
sales[product: 'Widget'][:revenue]           # chained

```

Also introduced `to_a` for extracting data as an array of hashes, the `Row` class for formula resolution, and `attr_accessor :data, :formulae`.

0.2.0 (2026-04-14): Enumerable

`Namo` includes `Enumerable`. The `each` method yields `Row` objects (not raw hashes), so `Enumerable` methods like `map`, `select`, `reduce`, `min_by`, and `flat_map` all see formulae as though they were data.

```

sales[:revenue] = proc{|r| r[:price] * r[:quantity]}
total = sales.reduce(0){|sum, row| sum + row[:revenue]}
cheapest = sales.min_by{|row| row[:price]}

```

Selection logic moved from `Namo` to `Row#match?`. Added `Row#to_h`. `each` returns an `Enumerator` when no block is given.

0.3.0 (2026-04-15): Contraction

Contraction via negated symbols in `[]`. The unary minus on a symbol (`-:price`) produces a `NegatedDimension`, which `[]` recognises as "remove this dimension from the result." The complement of projection — projection says what to keep, contraction says what to remove.

```

sales[-:price, -:quantity]          # remove dimensions
sales[-:price, -:quantity, product: 'Widget'] # contraction + select

```

Raises `ArgumentError` when mixing projection and contraction in the same call. Formulae carry through contraction. `NegatedDimension` and `Symbol#-@` introduced as supporting classes.

`Row` extracted to its own file (`Namo/Row.rb`). Tests split into per-class files.

0.4.0 (2026-04-15): Concatenation and row removal

The first two row-axis set operators.

`+` concatenates two `Namos` with matching dimensions. Appends rows from the second to the first. Formulae

merge — self's formulae take priority on conflict.

- removes exact matching rows. Whole-row set difference. If a row appears in the right operand, it's removed from the left. Both operators raise `ArgumentError` when dimensions differ.

```
jan_sales + feb_sales      # more rows, same dimensions
all_sales - returned_items # fewer rows, same dimensions
```

0.5.0 (2026-04-16): Intersection, union, symmetric difference

The remaining row-axis set operators, completing the set algebra.

& returns rows present in both operands. | returns all rows deduplicated (implemented via `Array#|`). ^ returns rows in one operand but not both — the symmetric difference.

```
sales & promotions      # rows in both
sales | new_arrivals    # all rows, no duplicates
sales ^ competitor     # rows unique to each side
```

All three require matching dimensions, raise `ArgumentError` otherwise. Formulae carry through from self, merge from other, self wins on conflict. | and ^ merge formulae from both sides.

Summary

The set operators (+, -, &, |, ^) together with selection, projection, contraction, and formulae give Namo a complete vocabulary for working with a single dataset or combining datasets that share the same dimensions. The next phase (0.6.0+) extends this to datasets with different dimensions via composition operators and adds richer selection and formula capabilities.

0.6.0: Comparisons

Row-level subset/superset tests using Ruby's comparison operators.

- `a < b` — a's rows are a strict subset of b's rows
- `a <= b` — a's rows are a subset of b's rows
- `a > b` — a's rows are a strict superset of b's rows
- `a >= b` — a's rows are a superset of b's rows
- `a == b` — same rows

These pair with the set operators algebraically: if `a & b == a`, then `a <= b`. Dimension and coordinate comparison is already available via `a.dimensions == b.dimensions` and `a.coordinates[:dim] == b.coordinates[:dim]`.

0.7.0: Proc-based selection

Extend [] to accept procs as selection predicates on dimensions.

```
namo[pe: ->(v){ v && v < 15 }]  
namo[price: ->(v){ v > 10.0 }, symbol: ->(v){ v != 'TEST' }]
```

Implementation: add a `when Proc` branch to `Row#match?` that calls the proc with the dimension value. Single addition to existing selection logic.

This enables multi-factor screening in one expression:

```
namo[pe: ->(v){ v && v < 15 }, price_to_book: ->(v){ v && v < 1.5 }]
```

Proc-based selection composes with contraction and projection in a single [] call.

0.8.0: Composition operators (*, **, /)

The dimensional composition algebra.

* (equi-join on shared dimensions)

Identifies shared dimension names between two Namos. Pairs rows where coordinates match on all shared dimensions. Non-shared dimensions extend the result.

```
ohlcvs * fundamentals # joins on exchange, symbol
```

** (Cartesian product)

Every row from the left paired with every row from the right. No automatic matching. The "explosive" operator — more sigil, more output.

** without constraints produces the full Cartesian product. * is derivable from ** — it's ** with shared-dimension filtering applied automatically.

/ (decomposition)

The inverse of *. Factors out dimensions.

```
combined / ohlcvs # removes dimensions exclusive to ohlcvs, keeps shared +
```



```
(combined / ohlcvs) * ohlcvs reconstructs combined.
```

0.9.0: Blocks on composition and set operators

Blocks on * and **

Both * and ** gain optional blocks for custom matching logic beyond exact dimension matching.

For *, the block receives a row from the left and the pre-filtered candidates from the right (already matched on shared dimensions):

```
ohlcv.*(fundamentals) do lrow, candidates|
  candidates.select{|f| f[:quarter_end] <= row[:date]}.max_by{|f| f[:quarter_end]}
end
```

The block can be passed as a named proc:

```
MOST_RECENT_QUARTER = proc do lrow, candidates|
  candidates.select{|f| f[:quarter_end] <= row[:date]}.max_by{|f| f[:quarter_end]}
end

ohlcv.*(fundamentals, &MOST_RECENT_QUARTER)
```

For **, the block receives a row and ALL rows from the right (no pre-filtering):

```
ohlcv.**(fundamentals) do lrow, candidates|
  # full control, including symbol matching if desired
end
```

** with a block that manually matches on shared dimensions produces the same result as * without a block. * is sugar on top of **.

Blocks on set operators

All set operators (+, -, &, l, ^) gain optional blocks that relax the matching condition from exact row equality to a custom predicate.

```
# Remove by symbol match rather than exact row match
today.-(exclusions){|a, b| a[:symbol] == b[:symbol]}
```

```
# Add candidates not already held
portfolio.+(new_candidates){!existing, candidate | candidate[:symbol] != ex
```

| with a block uses the same semantics as + with a block, then deduplicates.

0.10.0: Finite module and two-arity formulae

Finite module

A new module that includes Enumerable and adds `last` and `reverse_each` for finite collections. Namo includes Finite instead of Enumerable. Default implementation uses `entries`; Namo overrides for performance by going straight to `@data`.

```
module Finite
  include Enumerable

  def last(n = nil)
    n ? entries.last(n) : entries.last
  end

  def reverse_each(&block)
    return enum_for(:reverse_each) unless block_given?
    entries.reverse_each(&block)
  end
end
```

Finite is a standalone concept — potentially a separate gem. Any finite Enumerable can include it.

Namo includes Finite and overrides `last` to go straight to `@data`, wrapping results in Rows and avoiding the `entries` materialisation:

```
def last(n = nil)
  if n
    @data.last(n).map{|row| Row.new(row, @formulae, self)}
  else
    Row.new(@data.last, @formulae, self)
  end
end
```

Two-arity formulae

Row gains a reference to its parent Namo. Procs with arity 2 receive (row, namo), enabling cross-row computation:

```
e[:sma_20] = proc do |row, namo|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(20)
  window.sum{|r| r[:close]} / window.length.to_f
end
```

The `each` method passes `self` (the Namo) to Row:

```
def each(&block)
  return enum_for(:each) unless block_given?
  @data.each{|row_data| block.call(Row.new(row_data, @formulae, self))}
end
```

Row's constructor gains an optional Namo reference:

```
class Row
  def initialize(row, formulae, namo = nil)
    @row = row
    @formulae = formulae
    @namo = namo
  end
end
```

The `namo` parameter defaults to `nil` for backward compatibility — existing code that calls `Row.new(row, formulae)` keeps working. Row-scoped formulae never touch `@namo`. Two-arity formulae use it when present. If a Row is constructed without a Namo and a two-arity formula is called, the `nil` produces a clear error rather than a missing argument error.

Note: the current Row constructor (0.5.0) takes (row, formulae). Adding `@namo` is a prerequisite for both `Finite#last` wrapping Rows correctly and for two-arity formulae. This change is introduced alongside the `Finite` inclusion.

`Row#[]` dispatch for arity 1 and 2: - 1: `proc.call(row)` — row-scoped - 2: `proc.call(row, namo)` — collection-scoped

0.11.0: Parameterised formulae

Procs with arity > 2 receive (row, namo, *extra_args). Row#[] forwards extra arguments:

```
e[:sma] = proc do |row, namo, field, period|  
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(period)  
  window.sum{|r| r[field]} / window.length.to_f  
end
```

```
row[:sma, :close, 20] # Row inserts self and namo, forwards :close and 20
```

Row#[] dispatch extended: - 1: proc.call(row) — row-scoped - 2: proc.call(row, namo) — collection-scoped - >2: proc.call(row, namo, *extra_args) — parameterised

1.0.0: Stable release

The 1.0 release includes everything through 0.11.0: comparisons, proc-based selection, composition operators (*, **, /), blocks on all operators, Finite module, two-arity formulae, and parameterised formulae. This is the correct, tested, conservative foundation. No metaprogramming magic, no method_missing, no instance_eval. Formulae work via e[:name] = proc{|row| row[:close] / row[:book_value]} — clear, explicit, proven.

1.0 is the set of features that are well-understood, thoroughly tested, and unlikely to change.

Estimated performance: ~0.3s for a 2,000 row daily trading screen with indicators and scoring. Pure Ruby, no native dependencies. Adequate for interactive use and daily batch jobs. Not suitable for datasets over ~50,000 rows without patience.

1.1: Benchmarking suite

Performance and stress-testing infrastructure. Establishes baselines for every feature so that subsequent optimisations can be measured against concrete numbers.

The suite should cover:

- Construction: Namo.new from N rows, measuring hash ingestion and dimension/coordinate inference
- Selection: single value, array, range, proc, at various dataset sizes
- Projection and contraction: [] dispatch overhead
- Formulae: row-scoped resolution, formula chains (A references B references C), parameterised formulae
- Enumerable: each, map, select, reduce, max_by — measuring Row object creation per iteration
- Set operators: +, -, &, |, ^ at various dataset sizes
- Composition: * across different dimension overlaps

Each benchmark at multiple scales — 100, 1,000, 10,000, 100,000, 1,000,000 rows — to show where curves bend and where Ruby's overhead dominates.

Use benchmark-ips for iterations-per-second measurements. Results should be recorded and versioned so regressions are visible across releases.

The 1.1 numbers become the baseline for 2.x comparisons.

1.2: Loaders

Optional requires for common data sources. No loader loaded by default. No dependencies added to Namo core.

```
require 'namo/loaders/csv'
require 'namo/loaders/sequel'
require 'namo/loaders/json'
```

Each extends Namo with a class method (`from_csv`, `from_sequel`, `from_json`). Loaders are sugar — the constructor already takes arrays of hashes, so loading is always possible without them.

2.x: Bare names and Ruby-side optimisation

Theme: the expressive leap.

Estimated performance: `method_missing` on Row adds ~5-10% overhead versus 1.x on first access per dimension per Row. The `DefineAccessors` optimisation (below) recovers most of this by defining real methods on first access. After warm-up, 2.x should be within ~2-3% of 1.x. Pure Ruby performance tuning against the 1.1 benchmarking suite may yield further gains. All estimates — actual numbers depend on the 1.1 baseline.

From 2.0, `require 'namo'` loads bare name resolution by default. The full experience is the default. Users who want the 1.x explicit style can opt out:

```
require 'namo'           # 2.x default: bare names included
require 'namo/core'      # opt out: 1.x explicit style, no method_missing o
```

Bare name resolution

Row resolves dimension names and formulae as bare method calls via `method_missing`, delegating to `self[name, *args]`. This eliminates hash access syntax from formulae:

```
# 1.x style
e[:price_to_book] = proc{|row| row[:close] / row[:book_value]}

# 2.x with bare names via `method_missing` on Row
e[:price_to_book] = proc{|row| row.close / row.book_value}

# 2.x with bare names via `instance_eval` for arity-0 procs
```

```
e[:price_to_book] = proc{ close / book_value }
```

Bare names also work in cross-row formulae. Fixed dimension references resolve against the current Row. Variable field names (passed as parameters) use `Symbol#to_proc` via `&field`:

```
e[:sma] = proc do lrow, namo, field, period|
  window = namo[symbol: symbol, date: ..date].last(period)
  window.sum(&field) / window.length.to_f
end
```

Here `symbol` and `date` are bare names resolving against the Row. `&field` uses `Symbol#to_proc` to send the field name to each Row in the window.

Resolution order in `Row#method_missing`: formulae first, then row data, then super (raises `NoMethodError` for typos).

method_missing on Namo for formula assignment

Namo resolves formula assignment as bare method calls via `method_missing`, catching `name=` calls:

```
e.price_to_book = proc{ close / book_value }
e.sma = proc do lrow, namo, field, period|
  # ...
end
```

Module-based formula libraries (documented pattern)

With bare name resolution in place, formulae can be defined as plain Ruby methods in modules and included into Namo subclasses. This is not a Namo feature — it's standard Ruby. It's documented here as a recommended pattern:

```
module Indicators
  def sma(row, namo, field, period)
    window = namo[symbol: symbol, date: ..date].last(period)
    window.sum(&field) / window.length.to_f
  end

  def change
    close - open
  end
end
```

```

    def earnings_yield
      eps / close
    end
  end

  module Scoring
    def value_score
      return 0 unless pe && pe > 0
      pe < 10 ? 2 : pe < 15 ? 1 : pe < 25 ? 0 : -1
    end

    def total_score
      value_score + book_score + yield_score + momentum_score + trend_score
    end

    def action
      s = total_score
      s >= 5 ? 'BUY' : s >= 2 ? 'WATCH' : s >= 0 ? 'HOLD' : total_score <= -
    end
  end

  class TradingAnalysis < Namo
    include Indicators
    include Scoring
  end

```

Scoring strategies are swappable via dependency injection.

DefineAccessors optimisation

Inspired by Hashie's pattern — lazily define real methods on first access to avoid repeated `method_missing` overhead:

```

class Row
  def method_missing(name, *args)
    if @formulae.key?(name)
      define_singleton_method(name) do |*a|

```

```

        resolve(name, *a)
    end
    send(name, *args)
  elsif @row.key?(name)
    define_singleton_method(name) { @row[name] }
    @row[name]
  else
    super
  end
end
end
end

```

First access fires `method_missing` and defines a real method. Every subsequent access is a direct method call. Consider when profiling shows `method_missing` as a bottleneck.

Hashie as prior art, not a dependency

Hashie (particularly `Hashie::Mash`) is the primary prior art for method-style hash access and coercion in Ruby. Namo's Row and the coercion module design draw from Hashie's patterns. However, Hashie should not be used as a dependency or as internal storage for Namo:

- Namo already handles method-style access through Row's `method_missing`. Wrapping internal hashes in `Hashie::Mash` would add a second `method_missing` layer — double the dispatch overhead for no additional capability.
- Hashie does key normalisation (string/symbol indifference) and deep conversion of nested hashes. Namo doesn't need either — keys are always symbols and data should be flat.
- Hashie has a documented history of subtle breakage, particularly around `respond_to_missing?` interactions with other gems and method name collisions with Ruby built-ins.
- The performance cost of Mash allocation over plain Hash allocation is measurable.

Pure Ruby performance tuning

Profile against the 1.1 benchmarking suite. Identify and address hot paths within pure Ruby: Row object allocation, formula chain resolution, selection dispatch. No native code — just better Ruby.

Finite as a separate gem

Extract the Finite module (introduced in 0.10.0) into a standalone gem for use by any finite Enumerable, independent of Namo.

3.x: DSL, columnar storage, and C acceleration

Theme: the optimised engine.

Estimated performance: columnar storage accelerates selection-heavy workloads — scanning one contiguous

array versus touching every row's hash. C extension via RubyInline targets the remaining hot paths (iteration, row creation). Combined, the 2,000 row daily screen should drop from ~0.3s to ~0.1s. Selection on large datasets (100,000+ rows) should see the biggest improvement from columnar storage. The C extension adds ~4x speedup on tight numeric loops but the bottleneck is object allocation, not arithmetic — expect modest overall gains from C alone. All estimates.

From 3.0, require 'namo' loads both bare names and the DSL block syntax by default. Users can opt out at any granularity:

```
require 'namo'           # 3.x default: bare names + DSL
require 'namo/core'      # 1.x explicit style only
require 'namo/bare_names' # bare names without DSL
require 'namo/dsl'       # DSL block syntax without bare names
require 'namo/dsl/bare_names' # both (same as default, explicit about it)
```

The DSL and bare names are independent features on different objects — the DSL uses `method_missing` on a builder to catch formula registrations, while bare names use `method_missing` on `Row` to resolve dimension access. The DSL block works without bare names (procs use explicit `row[:close]` inside), and bare names work without the DSL block (formulae registered via `e.name = proc{}`). In practice, most users will want both.

DSL block syntax

A minimal-ceremony specification syntax using a builder with `method_missing`:

```
namo do
  sma          ->(field, period) { window(field, period).mean }
  change       -> { close - open }
  price_to_book -> { close / book_value }
  earnings_yield -> { eps / close }
  golden_cross  -> { sma(:close, 20) > sma(:close, 50) }
  value_score   -> { pe < 10 ? 2 : pe < 15 ? 1 : pe < 25 ? 0 : -1 }
  total_score   -> { value_score + book_score + yield_score + momentum_
  action        -> { total_score >= 5 ? 'BUY' : total_score >= 2 ? 'WAT'
end
```

The builder's `method_missing` catches each name-lambda pair. No `def`, no `end`, no `module`, no `class`, no `e.name = proc`. Just names and computations.

Columnar storage

A columnar backend (hash of arrays) to accelerate selection and aggregation for pure-Ruby workloads. The

design uses a mezzanine pattern — Namo delegates to `Namo::RowStore` or `Namo::ColumnStore` with the same public API.

Development order:

1. Default (current): always `RowStore`, no choice, no configuration
2. Choose at instantiation: Namo detects the input shape (array of hashes vs hash of arrays) or accepts an explicit `storage:` keyword
3. Dynamic: one primary layout with the other cached on demand, cache invalidated on mutation

```
# Row-oriented (array of hashes, current)
```

```
namo = Namo.new([symbol: 'BHP', close: 42.5], ...)]
```

```
# Column-oriented (hash of arrays)
```

```
namo = Namo.new({symbol: ['BHP', ...], close: [42.5, ...]})
```

When DuckDB is the backend (4.x), neither layout matters — the data lives in DuckDB. Columnar storage only matters when Namo is doing the computation itself in pure Ruby.

C extension via **RubyInline**

Optional C acceleration for hot paths via `RubyInline` (zenspider). Targets: selection (linear scan over contiguous array), iteration (row creation), and storage layout. Formula resolution stays in Ruby.

Pure Ruby is the default. C extension is an optional `require`. The API doesn't change:

```
require 'namo'
require 'namo/ext' # optional C acceleration, falls back to pure Ruby if
```

Coercion modules

Two forms of coercion, both declared as includable modules:

Ingestion coercion — fix types from CSV/JSON:

```
module OHLCVTypes
  def self.included(base)
    base.coerce :date, Date
    base.coerce :close, Float
    base.coerce :volume, Integer
  end
end
```

end

Dimension alignment coercion — map dimensions for * composition:

```
module TemporalAlignment
  def self.included(base)
    base.coerce :date, to: :quarter do |date|
      "#{date.year}-Q#{(((date.month - 1) / 3) + 1)}"
    end
  end
end
```

Coercion modules can be placed on the analysis class, on a loader, or on an ad hoc instance. Namo doesn't have an opinion about where they live. Inspired by Hashie's coercion extensions, but applied at the dimensional level.

The two forms are distinguishable by signature: - coerce :date, Date — ingestion, fix the type of this dimension's values - coerce :date, to: :quarter do ... end — alignment, map this dimension to another for *

Nested data flattening

Data from JSON APIs and other sources may arrive with nested structures:

```
{symbol: 'BHP', fundamentals: {eps: 1.2, pe: 14.5}}
```

Nested values break Namo's foundational principle — every key is a dimension. The solution is flattening at ingestion via a coercion declaration:

```
base.coerce :fundamentals, extract: [:eps, :pe, :book_value]
```

Which pulls specified keys up to the top level and drops the wrapper. The result is a flat hash where everything is a dimension.

Conversion discovery on *

When * finds no exact dimension name match, it checks the class's coercion registry for declared conversion paths. This replaces the earlier respond_to? design with explicit, inspectable, testable declarations.

Co-incident: Python bridge Paths 1 and 2 (separate repo)

Developed in a separate namo-python repository. The Python user writes Namo's Ruby DSL inside a triple-

quoted string. The 3.x DSL block syntax is what they see:

```
from namo import Namo

analysis = Namo.define("""
    e.golden_cross    = proc{ sma(:close, 20) > sma(:close, 50) }
    e.earnings_yield = proc{ eps / close }
    e.action          = proc{ total_score >= 5 ? 'BUY' : total_score >= 2 ? 'HOLD' : 'SELL' }
""")

results = analysis.run(data)
```

Path 1 (shell out): Python wrapper writes the DSL to a temp file, invokes ruby via subprocess, parses JSON results from stdout. Requires separate Ruby installation. ~0.5s overhead per invocation.

Path 2 (rb_call): MessagePack RPC to a persistent Ruby process. Lower latency (~50ms per call), more complex setup. Still requires Ruby installed separately.

Both paths build against 3.x Ruby Namo. The Python user experiences the full expressiveness of the DSL syntax. Ships with the `Indicators` module as a freebie so Python users get immediate value.

4.x: mRuby, SQL pushdown, and DuckDB

Theme: the database-integrated engine.

Estimated performance: DuckDB pushdown is the architectural win. Window functions (SMA, EMA) on millions of rows execute at ~0.3s in DuckDB versus ~45s in pure Ruby. The 2,000 row daily screen drops to ~0.05-0.1s total — DuckDB handles the heavy computation, Ruby handles the scoring and selection on the small result set. For the full 8.8M row ASX history, DuckDB makes it feasible where pure Ruby never could. Competitive with Polars (~0.1s) on equivalent workloads. All estimates.

mRuby compatibility

Namo's core compiled under mRuby. This enables embedding Namu in other environments without a full CRuby installation. Connects to existing mRuby compatibility work (e.g. `stateful.rb` 2.1.0).

DuckDB integration

DuckDB as a high-performance backend via Sequel or the `duckdb` gem. Four paths were considered, ordered from most manual to most automatic:

Path A (rejected): Write raw SQL for large slabs of functionality. Rejected — means maintaining SQL alongside Ruby, and the two representations can drift apart.

Path D (immediate): Use Sequel to generate DuckDB window functions before Namu ingests the data. No

changes to Namo.

Path C (next): Formulae carry SQL metadata alongside the Ruby implementation. Namo detects the backend and inlines SQL where available:

```
module Indicators
  def sma(row, namo, field, period)
    window = namo[symbol: symbol, date: ..date].last(period)
    window.sum(&field) / window.length.to_f
  end

  SQL = {
    sma: ->(field, period) {
      "AVG(#{field}) OVER (PARTITION BY symbol ORDER BY date ROWS BETWEEN :
    }
  }
end
```



Path B (eventually): Namo generates SQL from formula definitions automatically.

Different Namos in the same composition can use different backends. OHLCV from DuckDB with SQL pushdown, fundamentals from Sequel, macro data from CSV. They compose via * regardless of origin.

SQL pushdown from formula metadata

Formulae declare their SQL equivalents. Namo uses the SQL version when a database backend is detected and falls back to Ruby when it can't. The Ruby implementation is the truth. The SQL is the optimisation. Both are declared in the same place.

Co-incident: Python bridge Path 3 (separate repo)

With mRuby compatibility in 4.x, the Python bridge can embed the mRuby interpreter directly. `pip install namo` works with no separate Ruby dependency. Built in the `namo-python` repo against the 4.x mRuby-compatible core.

Near-zero call overhead — in-process, no serialisation. Cross-platform binary distribution needed (wheels per platform).

5.x: Streaming, transpilation, and genetic algorithms

Theme: the analytical platform.

Estimated performance: streaming eliminates the memory constraint — 8.8M rows never loaded simultaneously. Processing cost is per-window rather than per-dataset. Transpilation to SQL or Python/Polars delegates execution

entirely to native engines — Namo becomes a specification tool with near-zero runtime cost. GA support benefits from DuckDB's speed on each candidate evaluation. Performance is bounded by the backend, not by Namo. All estimates.

Streaming / progressive computation

For large datasets (8.8M+ rows), process data in windows rather than loading everything. Rolling buffer per symbol, incremental indicator computation, results retained, raw data discarded.

A streaming Namo stays unfrozen — it accepts new rows, invalidates caches on mutation, and reflects current state. The analytical case is a Namo that gets frozen after setup.

Transpilation

Namo as a specification language that emits executable code in other targets:

- SQL (DuckDB, PostgreSQL) for production deployment
- Python/Polars for handoff to Python teams

Explore interactively in Ruby. Deploy as transpiled output. The exploration tool and the production artifact are the same object viewed from different angles.

Genetic algorithm support

Parameterised formulae enable GA-based strategy optimisation:

```
namo do |params|
  value_score -> { pe < params[:pe_strong] ? 2 : pe < params[:pe_good] ? 1
  action      -> { total_score >= params[:buy_threshold] ? 'BUY' : 'HOLD'
end
```

Genome is a hash of parameters. Fitness is backtest return. Mutation/crossover operate on hashes. DuckDB evaluates each candidate at native speed.

Speculative / unversioned

These options are not scheduled. They become relevant when need and adoption justify the investment.

Rust acceleration via Rutie + Thermite

Rust accelerates specific hot paths within Ruby-side Namo. Ruby remains the host. The native code is an optimisation layer underneath the Ruby API. Formula resolution, `method_missing`, and DSL stay in Ruby.

Rutie provides Ruby bindings for Rust via macros. Thermite handles compilation and gem distribution with pre-compiled binaries. ~20x speedup observed on parsing benchmarks.

Targets: selection, iteration, storage layout. The same hot paths as the C extension (3.x) but with Rust's memory safety and the Rust ecosystem's momentum in data tooling (Polars, DuckDB, Apache Arrow).

```
namo/  
  lib/  
    namo.rb          # pure Ruby, always works  
    namo/ext.rb      # loads native extension, falls back to pure Ruby  
  ext/  
    namo_ext/  
      Cargo.toml     # Rust project  
      src/  
        lib.rs       # Rust implementations of hot paths  
  Rakefile           # Thermite tasks for building
```

Work on Rust acceleration builds familiarity and infrastructure that would feed into the Rust-native engine if it ever happens.

FFI (generic)

Compile any language (Rust, C, Crystal) as a C-compatible shared library (`cdylib`), load via Ruby's `ffi` gem or `stdlib Fiddle`. More boilerplate than Rutie but language-agnostic — the same FFI interface works regardless of what produced the shared library.

Rust-native engine with Ruby DSL parser

Namo's core reimplemented in Rust with a parser for the Ruby DSL syntax. The DSL specification strings are parsed, not evaluated — no Ruby runtime at all. The Rust engine is language-agnostic and can be exposed to any host language via its native FFI mechanism:

- Python via PyO3 (the same way Polars works)
- Ruby via Rutie (replacing pure Ruby internals with Rust for speed)
- Node.js via napi-rs
- Go via CGo
- Any language via C-compatible FFI

The Ruby DSL becomes a portable notation — a specification language that any host can submit to the Rust engine. The engine parses it, builds the internal representation, and executes it at native speed.

This path also connects to the transpilation story — if the Rust engine can parse the DSL, it can also emit SQL, Python/Polars expressions, or other output formats from the same parsed representation.

Not constrained to any single language. Largest engineering investment. Only justified if Namu achieves significant adoption across multiple language communities.

WASM compilation

Compile the Rust-native engine or mRuby core to WebAssembly for browser execution. Enables Namo-powered analysis in client-side web applications without a server.

Open questions

Mutability

Namo currently has `attr_accessor :data, :formulae`, making instances fully mutable. This conflicts with caching (stale results), thread safety (race conditions), and the principle that formulae are declarations.

The proposed model: Namos are mutable by default for interactive exploration. Call `freeze` when the shape is settled. Ruby's built-in `freeze` semantics apply — mutation methods (`[]`, `.name=`) check `frozen?` and raise `FrozenError` if so. Caching is safe on frozen Namos. Threads can share frozen Namos.

```
namo = TradingAnalysis.new(ohlcv_data)
namo.change = proc{ close - open }
namo.score = proc{ ... }

# exploration done
namo.freeze

# now cacheable, shareable, immutable
namo.score = proc{ ... } # => FrozenError
```

The streaming case is simply a Namo that never gets frozen. No special class needed — the distinction between analytical and streaming is a lifecycle difference, not a type difference. `freeze` is the transition point.

`attr_accessor` should become `attr_reader` on the public API, with mutation going through methods that check `frozen?`. Internal state (`@data`, `@formulae`) should be frozen when the Namo is frozen.

Live data and cache invalidation

For streaming/live Namos that stay unfrozen, formula results cached on Rows become stale when new data arrives. Options:

- No caching on unfrozen Namos (simple, slower)
- Generation counter — each mutation increments a counter, cached results carry the generation they were computed at, stale results are recomputed
- Event-based invalidation — mutation notifies dependents

The simplest approach (no caching when unfrozen) is probably right for the first implementation. Optimise if profiling shows it matters.

Coercion design

The coercion section (under 3.x) proposes a dual-purpose `coerce` keyword for both ingestion type fixing and dimensional alignment. Open questions:

- Is a single keyword for two different operations clear or confusing?
- Should ingestion coercion and alignment coercion be separate APIs?
- When multiple alignment coercions exist for a dimension (e.g. `:date` can coerce to both `:quarter` and `:month`), how does `*` choose? Raise ambiguity error? Accept a preference list?
- Should alignment coercion include a strategy (e.g. "most recent before") or just a value transformation?
- How does coercion interact with `freeze`? Are coercion declarations frozen with the `Namo`?
- Should nested data flattening (`extract:`) be part of the coercion API or a separate ingestion concern?
- Should flattening be recursive (nested hashes within nested hashes) or single-level only?

Formula shadowing and method collisions

If a formula and a data dimension share the same name, which wins in Row resolution? Currently formulae take priority. Should this be configurable? Should it warn?

Related: Hashie::Mash's long history of problems with method names that collide with Ruby built-ins (`class`, `hash`, `sort`, `zip`, `count`) is directly relevant. Row's `method_missing` has the same risk — a dimension named `class` or `method` would shadow Ruby's built-in methods. Options:

- Maintain a safelist of Ruby method names that `method_missing` won't intercept
- Warn when a dimension name collides with a built-in
- Always allow hash-style access (`row[:class]`) as a fallback when bare name access is blocked
- Document the risk and accept it — dimension names like `class` and `method` are unlikely in practice

Operator return types with subclasses

When `TradingAnalysis * Namu` is evaluated, what class is the result? `TradingAnalysis` (preserving the included modules)? `Namu` (the base class)? The left operand's class? This affects whether formulae from included modules are available on the composed result.

Presentation examples

`Namu`'s audience extends beyond traders and financial analysts. The design — type-agnostic dimensions, composable formulae, set algebra, bare name resolution — is relevant to anyone working with structured multidimensional data. A strong set of examples across disciplines demonstrates that `Namu` is a general tool, not a finance library.

Each example targets a different discipline and highlights a specific `Namu` capability. The companion document (Namu Presentation Examples with Comparisons) expands each example into a full four-stage progression (competitor tool → 1.x → 2.x → 3.x) with side-by-side code comparisons against `Pandas`, `Polars`, `xarray`, `R/dplyr`, and `Julia/DataFrames.jl`.

Finance / trading

Compose OHLCV with fundamentals via `*` with a temporal alignment block. Parameterised `sma` formula, scoring ladder, action decision. Key selling point: composable indicators, cross-source formulae, lazy dependency chains.

Climate / environmental science

Compose temperature readings with station metadata via `*`. Derive anomalies, classify extremes. Key selling point: one-operator composition of observational data with geographic metadata. Comparison: `xarray`.

Genomics / bioinformatics

Compose gene expression data with sample metadata. Fold change, significance screening, categorical classification. Key selling point: keyword selection on computed textual dimensions — `study[significant: true, category: 'upregulated']`. Comparison: `R/dplyr`.

Survey / social science

Compose survey responses with demographics. Categorical classification, string interpolation in formulae, textual selection. Key selling point: type agnosticism — every formula produces text, not numbers, and selection works identically on text dimensions. Comparison: `Polars`.

Sports analytics

Three-way composition: player stats, team data, match data via `stats * teams * matches`. Per-90-minute metrics, rolling form via parameterised SMA. Key selling point: three data sources composed with two operators. Comparison: `Pandas`.

Supply chain / logistics

Compose shipments with route costs. Cost, margin, profitability chain. Key selling point: contraction with `-:` to focus output — `logistics[profitable: false][-:shipment_id, -:weight, -:date]`. Comparison: `Julia/DataFrames.jl`.

Astronomy / physics

Compose stellar observations with catalogue data. Distance modulus formula using `Math.log10`, HR diagram classification. Key selling point: Ruby's full standard library inside `procs`, and `proc`-based selection predicates — `distance_ly: ->(v){ v < 500 }`. Comparison: `R`.

Presentation design principles

- Show all four stages in sequence: current tool, 1.x, 2.x, 3.x. The progression is the argument.
- Pause on 1.x — it's already better. Let the audience absorb that before showing bare names.
- The 1.x to 2.x transition is the biggest visual change. Give it a moment.
- Start with the audience's discipline. Breadth first, then depth.
- The selection line is the closer for every example.
- Have a live IRB session ready. The immediacy is more convincing than slides.
- Acknowledge where competitors are good. Honesty makes the wins more credible.